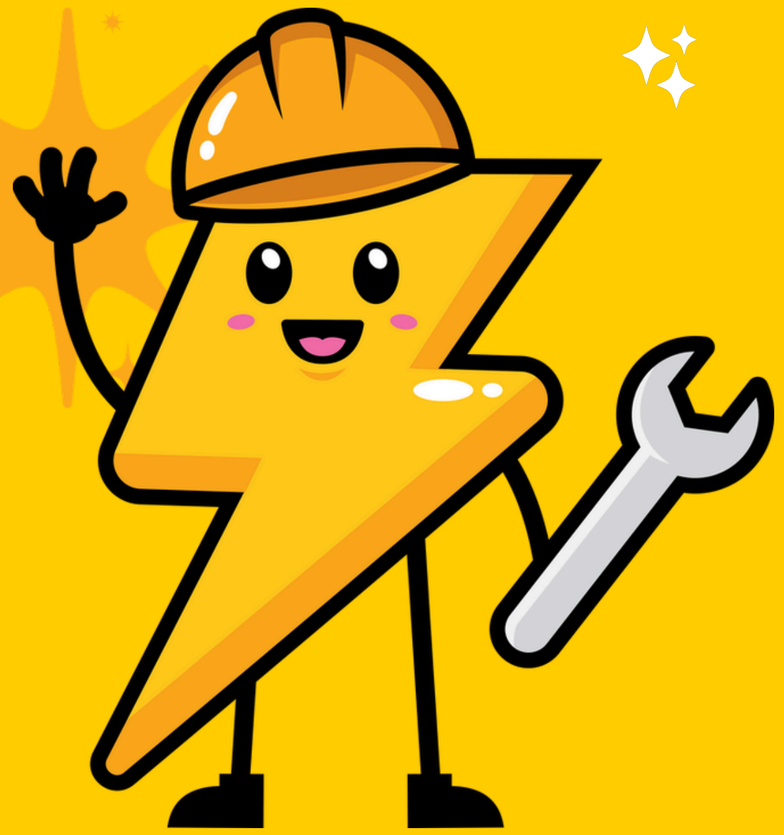
FIT3143 Parallel Computing | Lab #3, Week 12

EV CHARGING

A wireless sensor network in Open MPI + OpenMP

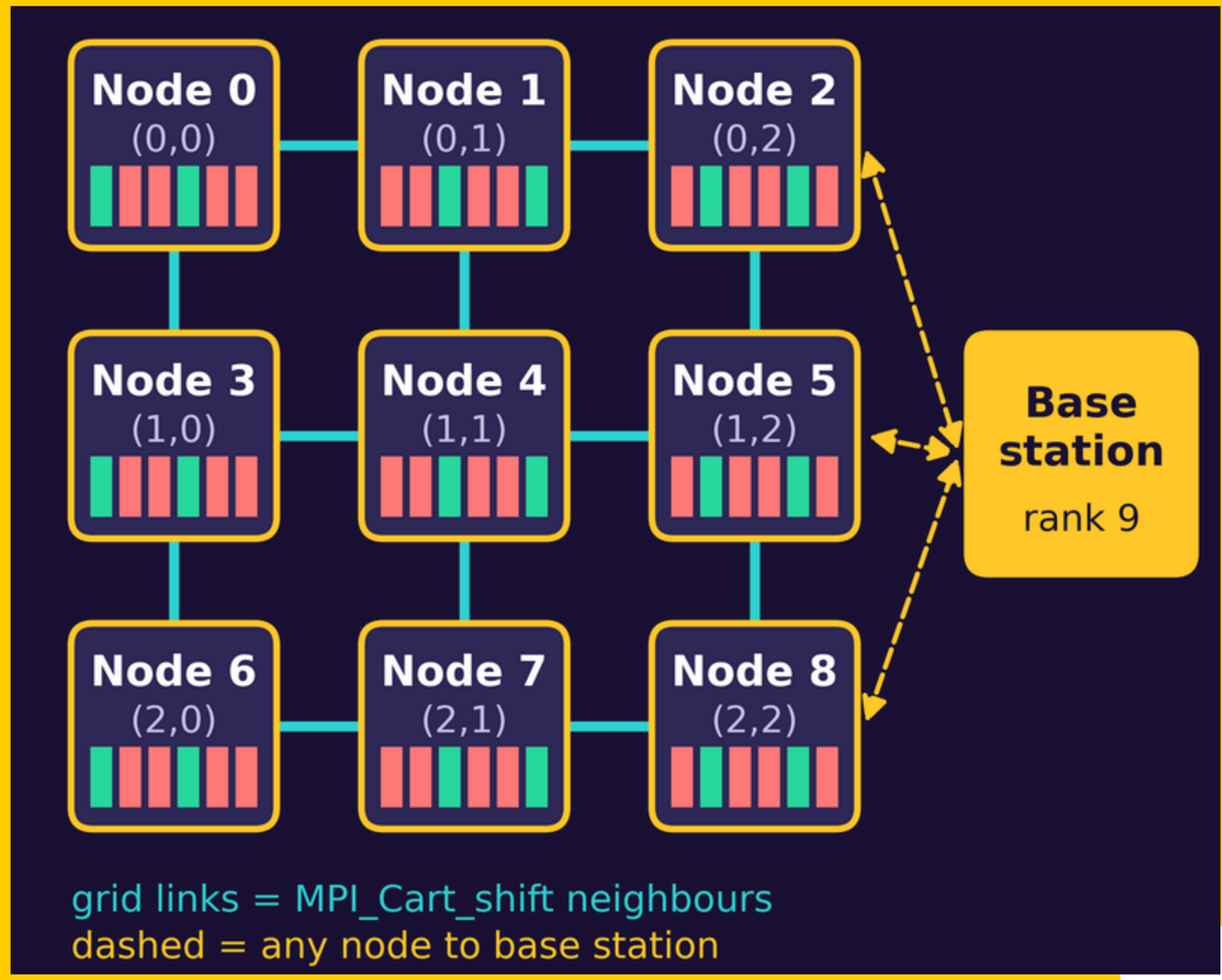


Presented By: Erwyna Soo Wen Xin (36555789)
Taabish Farooq Bhat (35473932)

ARCHITECTURE

Presenter: Taabish

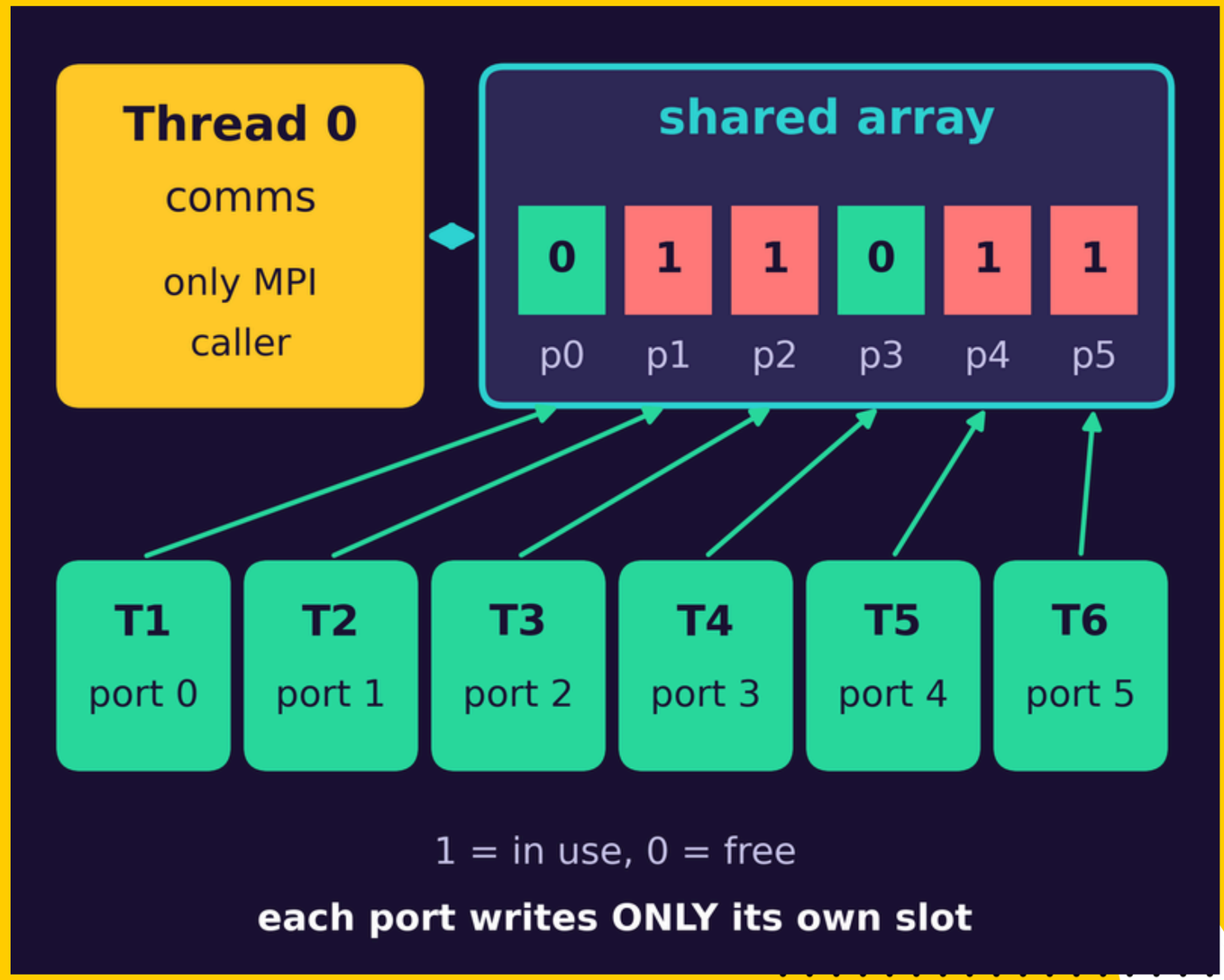
- One MPI process per charging node
- Base station: 1 extra process, the last rank
- Nodes form a rows x cols Cartesian grid
- A node talks only to up, down, left, right
- Any node can report straight to the base station



TASK 1: INSIDE A NODE

Presenter: Taabish

- OpenMP team of k + 1 threads
- Thread 0: the only MPI caller (FUNNELED)
- Threads 1 to k: one charging port each
- Shared array: 1 in use, 0 free
- Each port writes only its own slot: atomics, no mutex



TASK 1: VIRTUAL TOPOLOGY

Presenter: Taabish

- MPI_Comm_split: base station leaves the grid (MPI_UNDEFINED)
- MPI_Cart_create: rows x cols grid, no wrap at the edges
- MPI_Cart_shift: up, down, left, right; edges get MPI_PROC_NULL
- rows = 1 gives the line topology for Task 3b

main.c: base station out, grid in

```
67  MPI_Comm_split(MPI_COMM_WORLD, is_node ? 0 : MPI_UNDEFINED, world_rank, &node_comm);
75  int dims[2] = { cfg.rows, cfg.cols }, periods[2] = { 0, 0 };
76  MPI_Cart_create(node_comm, 2, dims, periods, 0, &grid_comm);
```

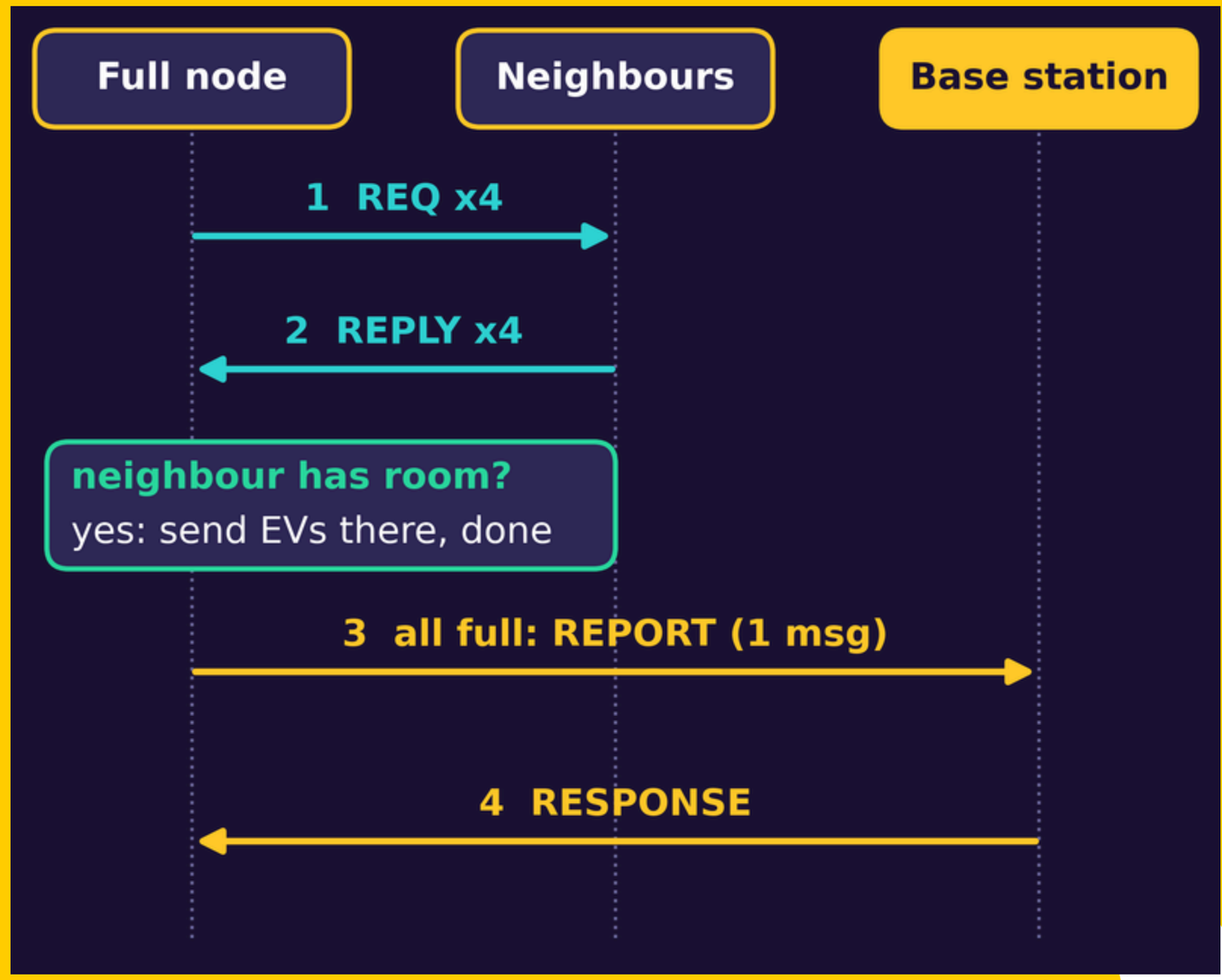
node.c: neighbours from the topology

```
695  MPI_Cart_shift(grid_comm, 0, 1, &n->nb_raw[0], &n->nb_raw[1]);
696  MPI_Cart_shift(grid_comm, 1, 1, &n->nb_raw[2], &n->nb_raw[3]);
```

TASK 1: WHEN A NODE FILLS UP

Presenter: Taabish

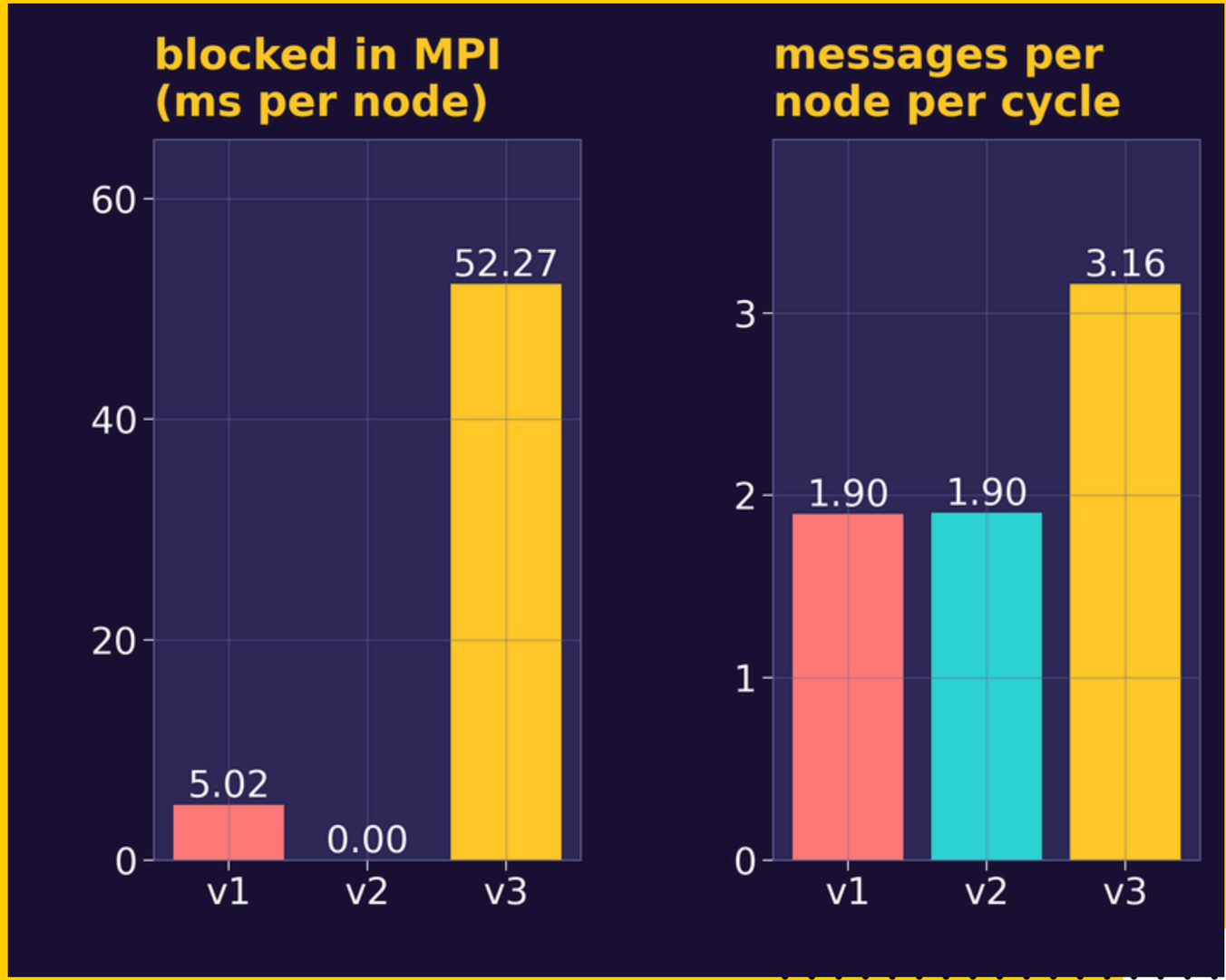
- Free ports <= threshold: ask the 4 neighbours
- A neighbour has room: send EVs there
- All full too: ONE report to the base station
- Report = 1 derived datatype, 176 B
- Time, node, neighbours, msgs, bytes



TASK 1: THREE NODE VERSIONS

Presenter: Taabish

- v1 blocking: waits in MPI_Probe, 5 ms blocked per node
- v2 nonblocking: Isend, Irecv, Testall, never blocks
- v3 Neighbor_allgather: 1.66x the messages, 52 ms blocked
- Same decision code in all three
- Chosen: v2



TASK 2: BASE STATION DECISION

Presenter: Erwyna

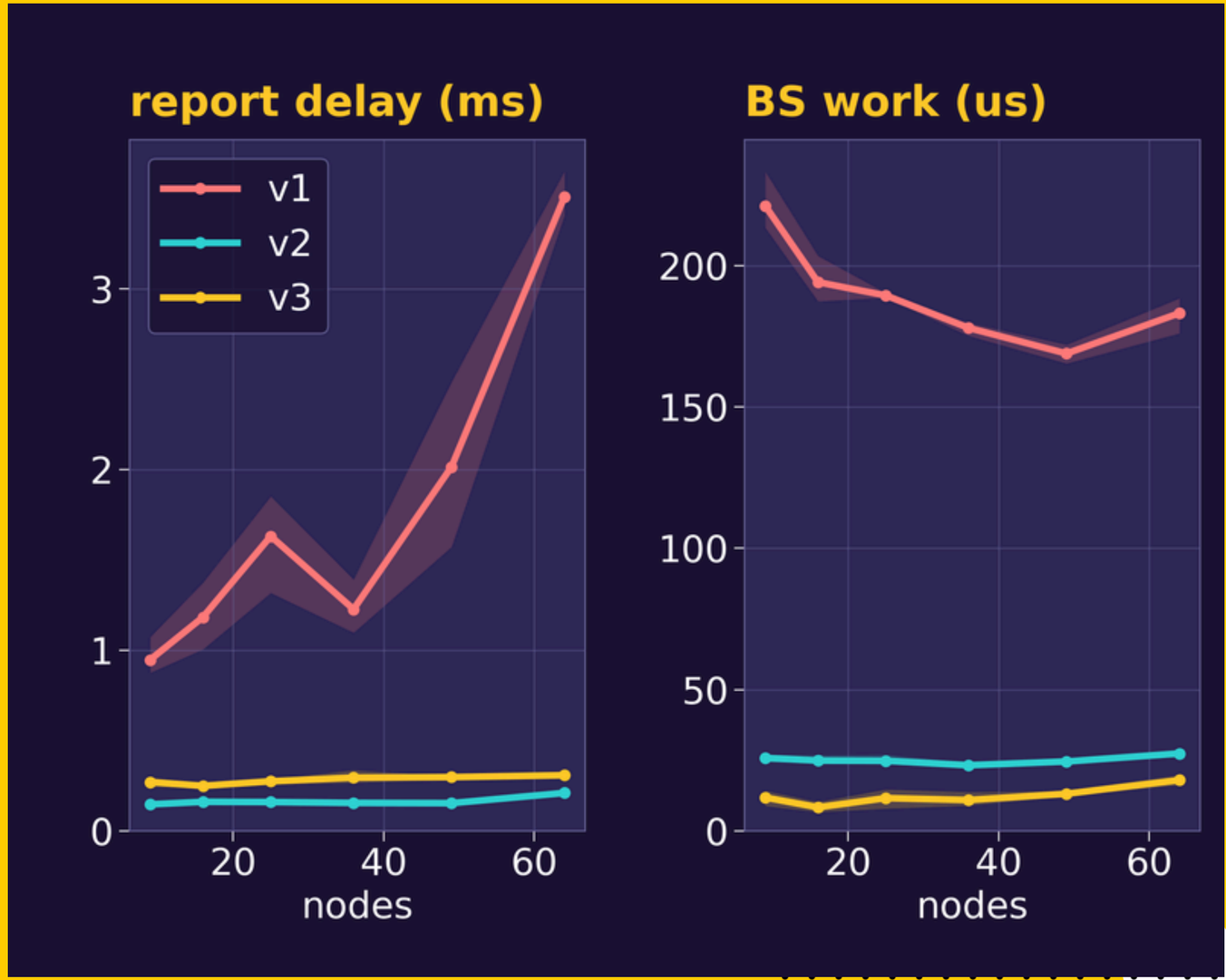
- 1 process, 1 thread does all MPI
- Node 0 reports; neighbours are 1 and 3
- Both reported inside window W: no nodes nearby
- Else: send ALL free nodes 2 hops away
- None free: widen to 3, 4, ... hops



TASK 2: THREE BASE STATION VERSIONS

Presenter: Erwyna

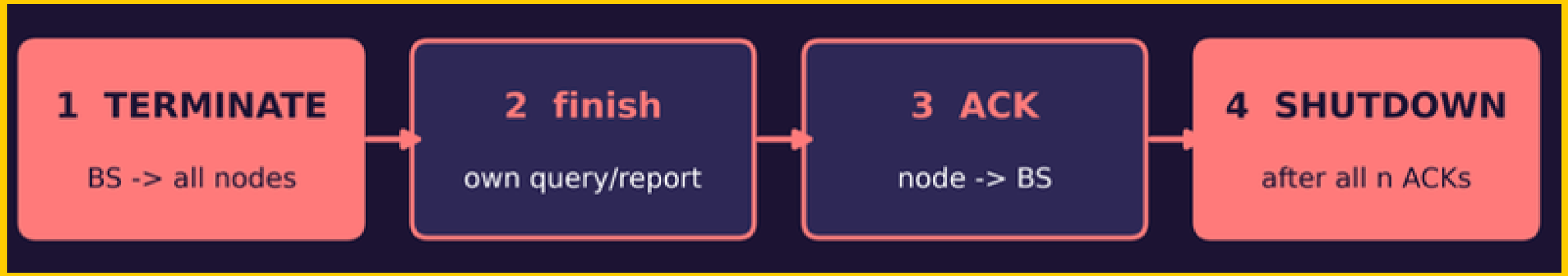
- v1 probe + blocking send: 0.95 to 3.5 ms, grows with nodes
- v2 one Irecv: fastest, 0.15 to 0.21 ms
- v3 persistent + Testsome: 0.27 to 0.31 ms, flat
- v3 does the least work per report
- Kept v3: half the work of v2



TASK 2: LOG AND GRACEFUL SHUTDOWN

Presenter: Erwyna

- Log per report: time, reporter, neighbours, msgs, bytes, answer
- End of run: sim time, alerts, messages, delay per message type
- Per node table: cycles, reports, round trips, time blocked
- Log writes go through a 1 MB buffer, not one disk write per line
- TERMINATE, wait for every ACK, then SHUTDOWN: nothing in flight



TASK 3: HOW WE MEASURED

Presenter: Erwyna

- Every message carries its send time
- One way delay = receive time minus send time
- Round trips use one rank's clock
- MPI_Wtime is not global here, so the system clock is used
- Uniform heavy load: 75% of ports busy

32 grid sizes

2x2 to 8x8: 4 to 64 nodes

32 line sizes

1x2 to 1x64

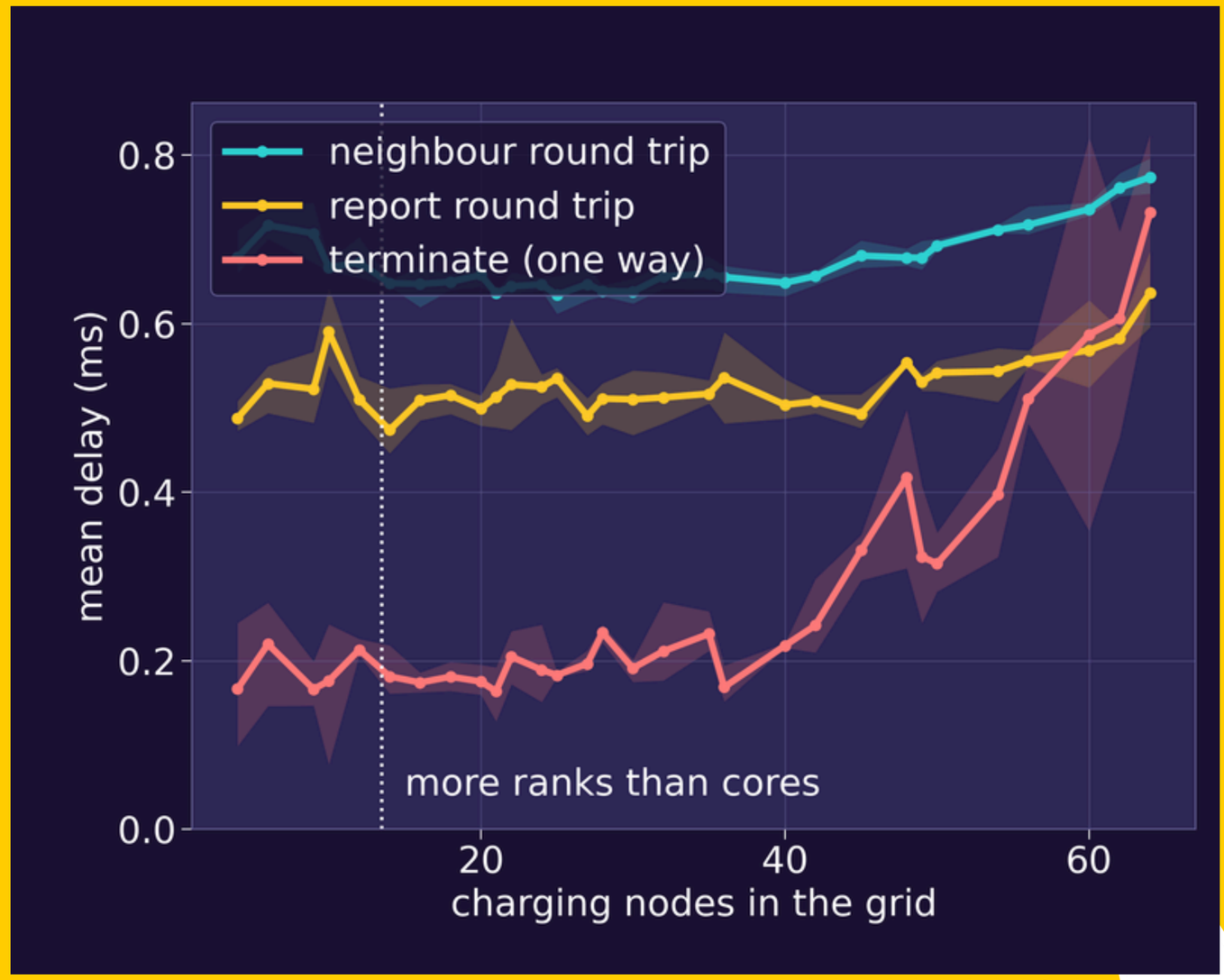
3 seeds x 3 versions

8 message types, 708 runs in total

TASK 3A: DELAY VS GRID SIZE

Presenter: Erwyna

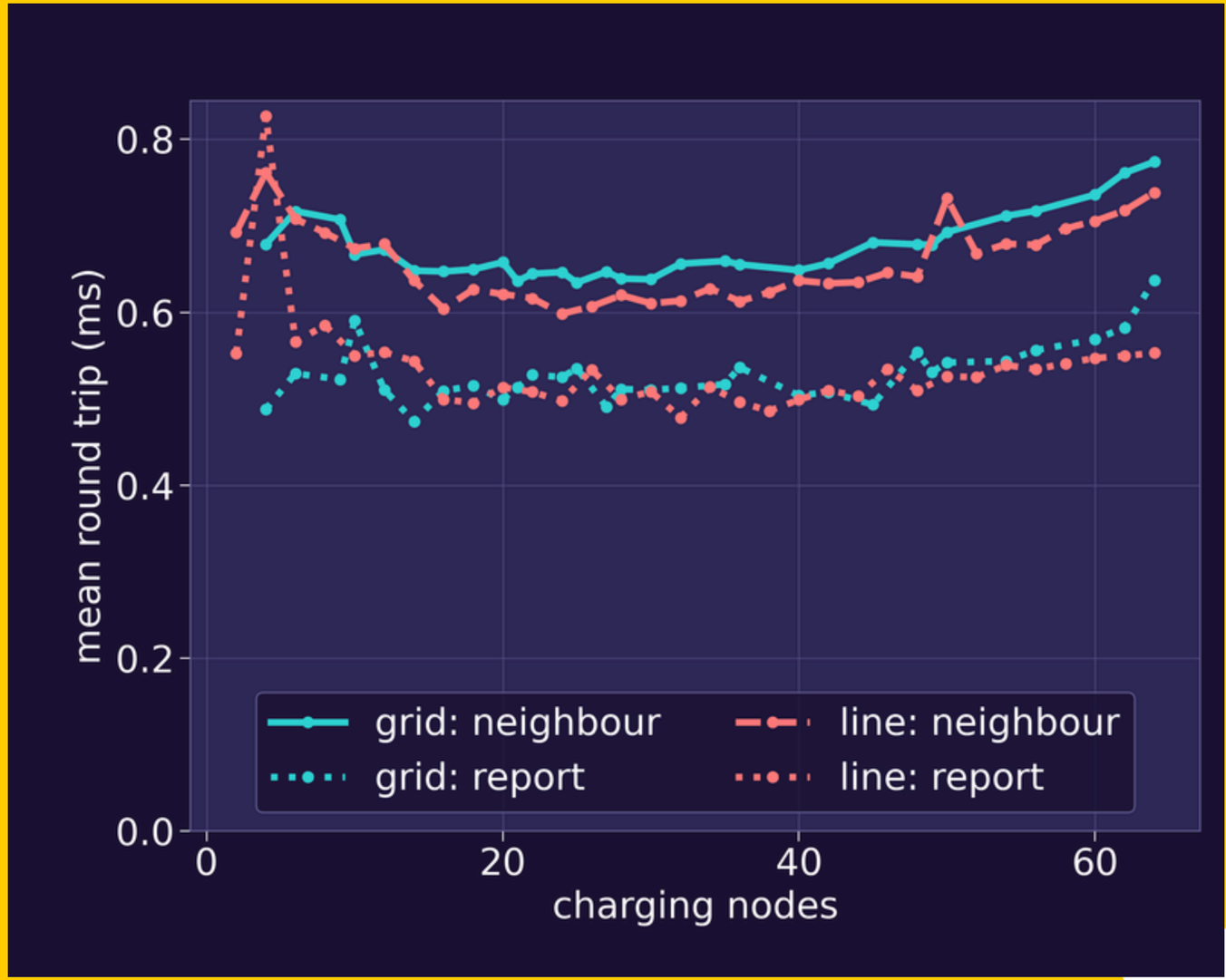
- Neighbour round trip 0.65 to 0.77 ms: near flat, never more than 4 neighbours
- Report round trip 0.49 to 0.64 ms: base station keeps up
- Terminate 0.17 to 0.73 ms: n separate sends, O(n)
- Past 13 nodes, ranks share 14 cores



TASK 3B: LINE TOPOLOGY

Presenter: Taabish

- Same code, rows = 1: 1x2 to 1x64
- At most 2 neighbours, not 4
- Neighbour round trip about the same: 0.66 vs 0.68 ms
- 2.4x the reports: 1.47 per node vs 0.60
- Both neighbours full more often, so more base station traffic



EXTRA: WINDOW AND POLLING

Presenter: Erwyna



- Longer window W: 'none nearby' answers rise from 2% to 18%, then level off
- No poll sleep: 0.17 ms round trip but 39 CPU s; our 200 us: 0.66 ms, 4 CPU s

LIMITATIONS AND FUTURE WORK

Presenter: Taabish

Limitations

- One base station: a single point of failure that every report goes through
- Laptop has 14 cores: past 13 nodes ranks share cores
- MPI assumes reliable links; real radios drop packets
- Random synthetic load, not real EV traces

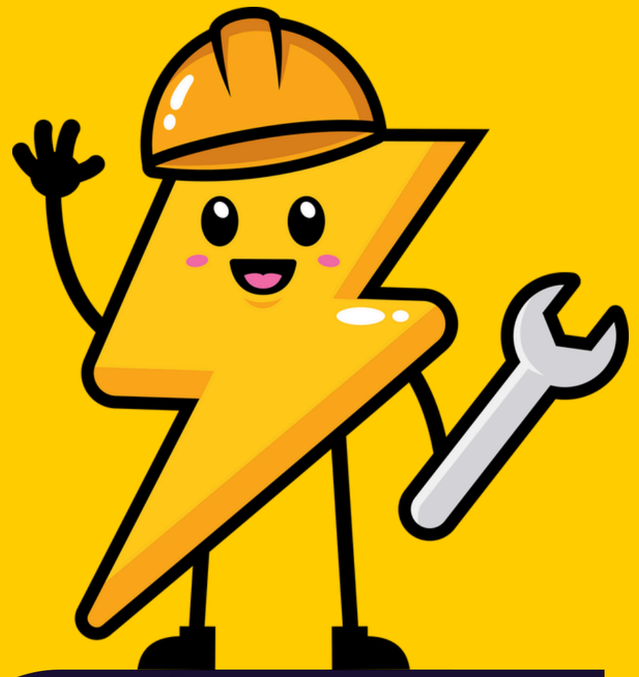
Future work

- Split the base station by region
- One collective for TERMINATE instead of n sends
- Poll adaptively: spin briefly, then sleep
- Acks and timeouts for lossy links
- Full sweep across CAAS machines

THANK YOU!
QUESTIONS?

Erwyna: Task 2 base station, Task 3a grid
Taabish: Task 1 nodes, Task 3b line

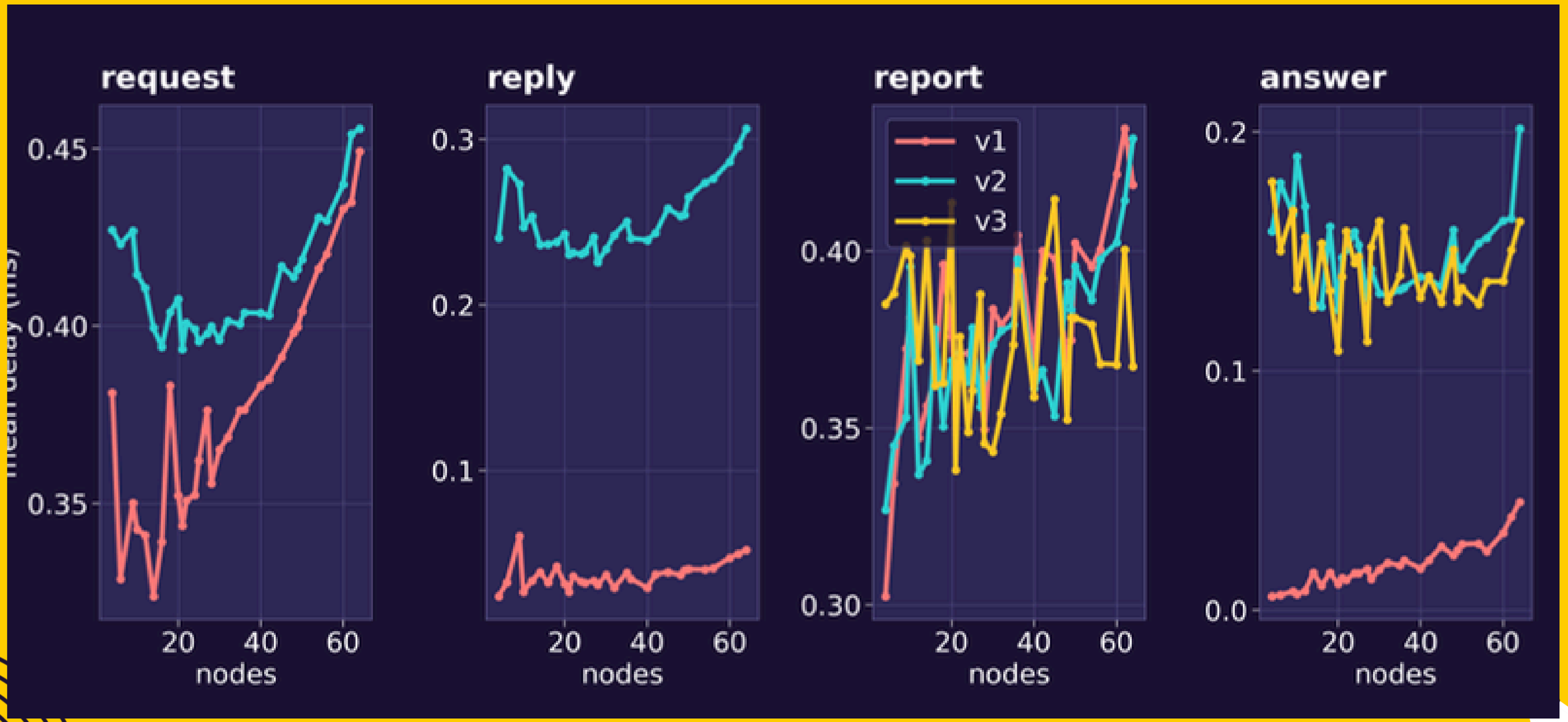
Appendix A1 to A8: every message type's graph, code, sample log, tests, CAAS.



FIT3143 Lab #3
Erwyna and Taabish
Monash Malaysia

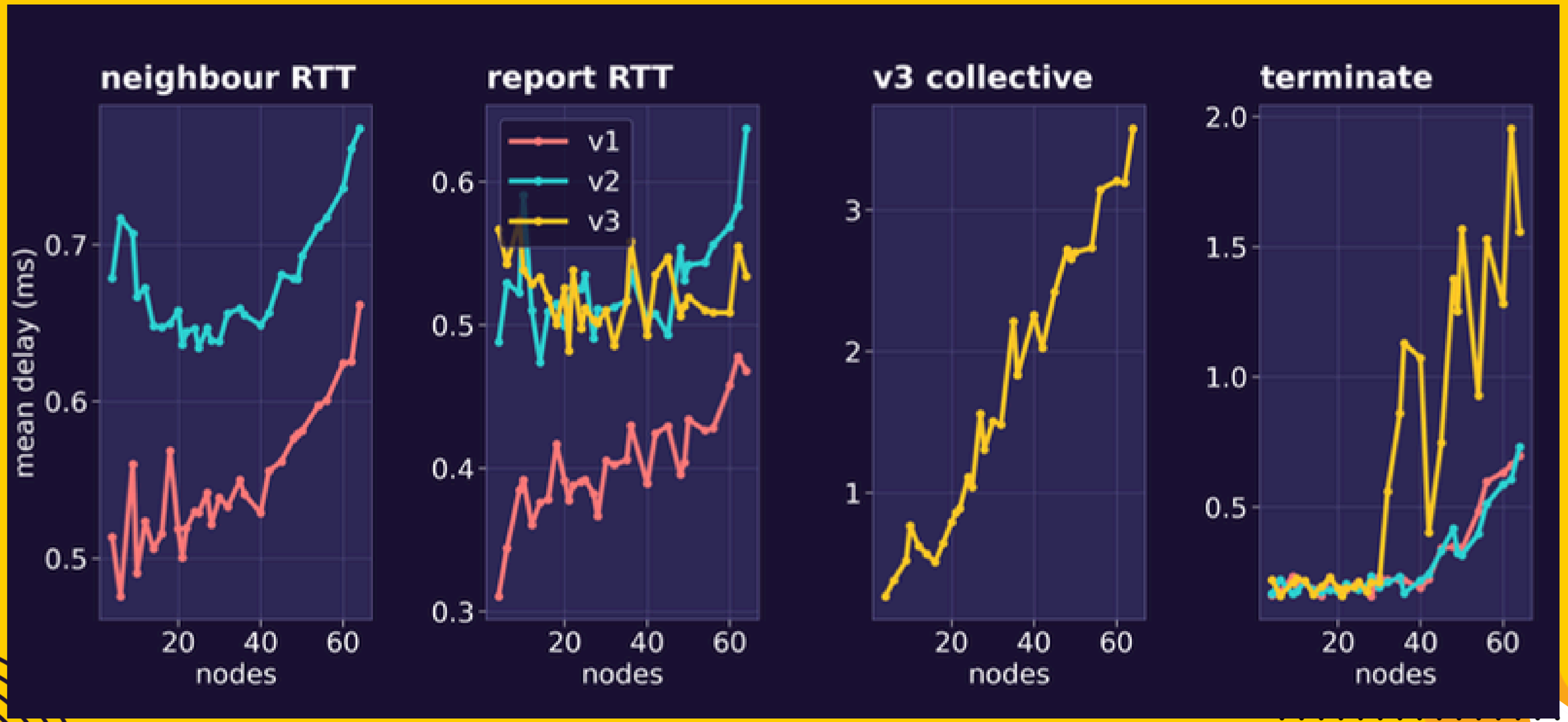
A1: GRID, ONE WAY DELAYS

Appendix (Q&A): 32 grid sizes, 3 seeds, node v1 red, v2 cyan, v3 yellow



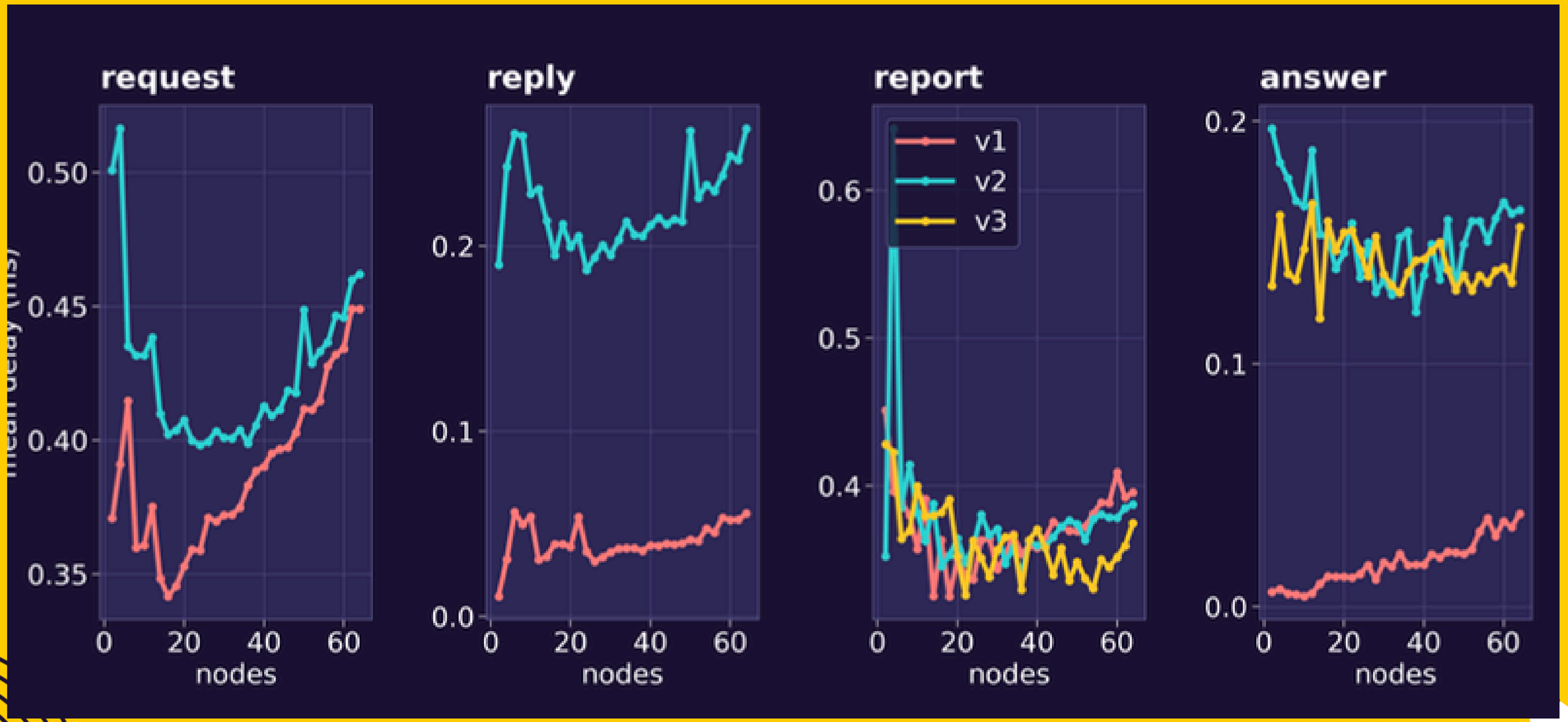
A2: GRID, ROUND TRIPS AND OTHERS

Appendix (Q&A): 32 grid sizes, 3 seeds, node v1 red, v2 cyan, v3 yellow



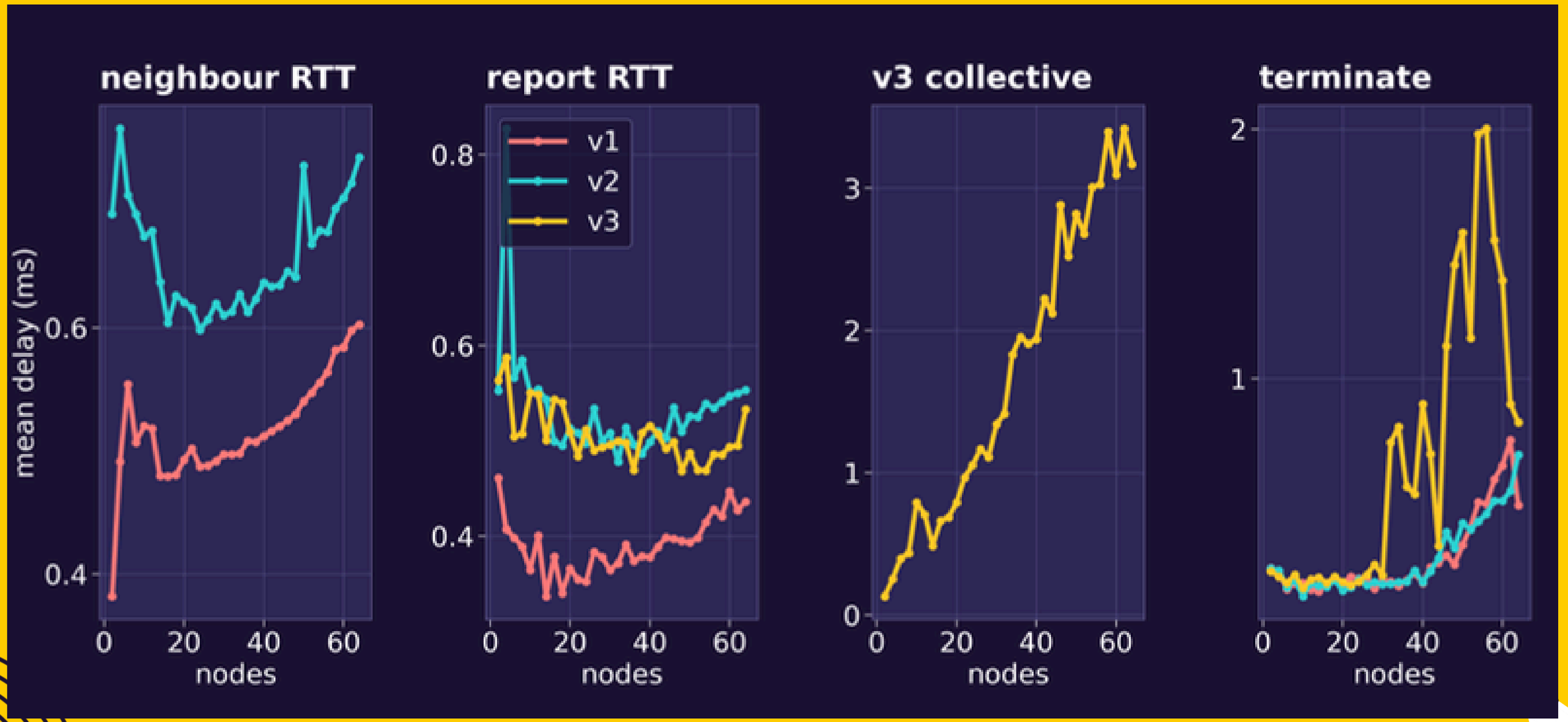
A3: LINE, ONE WAY DELAYS

Appendix (Q&A): 32 line sizes 1x2 to 1x64, 3 seeds, node v1 red, v2 cyan, v3 yellow



A4: LINE, ROUND TRIPS AND OTHERS

Appendix (Q&A): 32 line sizes 1x2 to 1x64, 3 seeds, node v1 red, v2 cyan, v3 yellow



A5: MORE CODE (Q&A)

Appendix: not presented, for questions

node.c: k ports + 1 comms thread

```
729  #pragma omp parallel num_threads(cfg->k + 1)
730  {
731  int tid = omp_get_thread_num();
737  comm_loop(n);
738  } else {
739  port_loop(n, tid - 1);
```

base_station.c: the window rule

```
135  if (n_nb > 0 && n_recent == n_nb) {
136  rs->status = RESP_NONE_NEARBY;
146  for (int d = 2; d <= max_d; d++) {
154  if (known_full(b, c, t, window)) continue;
```

- Top: the OpenMP team. Thread 0 runs every MPI call, threads 1 to k are the ports
- Bottom: all neighbours reported inside W, so none nearby; else walk the 2 hop ring
- Line numbers are from the submitted files

A6: NONBLOCKING CODE (Q&A)

Appendix: not presented, for questions

node.c, version 2: post receives, send, keep going

```
454  MPI_Irecv(&n->q_in[i], 1, n->tn, n->nb[i], TAG_REPLY, MPI_COMM_WORLD, &n->q_rreq[i]);
457  MPI_Isend(&n->q_out[i], 1, n->tn, n->nb[i], TAG_REQ, MPI_COMM_WORLD, &n->q_sreq[i]);
470  MPI_Testall(n->n_nb, n->q_rreq, &done, MPI_STATUSES_IGNORE);
```

base_station.c, version 3: persistent + Testsome

```
377  MPI_Recv_init(&b->rep_buf[i], 1, b->tr, i, TAG_REPORT, MPI_COMM_WORLD,
           &b->rep_req[i]);
386  MPI_Testsome(b->n, b->rep_req, &outcount, idx, MPI_STATUSES_IGNORE);
390  MPI_Start(&b->rep_req[idx[j]]);
```

- Node v2: receives are posted BEFORE the sends, so a reply always has a buffer
- BS v3: one armed receive per node; Testsome drains a whole burst in one call
- MPI_Start re-arms the same request: no set up cost per report

A7: Sample Base Station Log (Q&A)

logs/sample/base_station.log: one report as the base station logged it

```
[2026-10-16 23:25:32.280] REPORT #2 from node 4 (1,1), its report #2, cycle 3, sim t+0.3 s
  congestion found  2026-10-16 23:25:32.279
  reporter          free 1 of 6 ports (threshold 1), node version 2
  neighbours        1(0,1) free 1 | 7(2,1) free 0 | 3(1,0) free 0 | 5(1,2) free 1
  neighbour msgs    4 out (160 B), 4 in (160 B), exchange took 0.783 ms
  report            176 B, one way delay 0.311 ms
  window check      0 of 4 neighbours reported in the last 0.30 s [1:no 7:no 3:no 5:no]
  ANSWER            4 nearest available node(s), 2 hops: 0(0,0) free ? 2(0,2) free ?
      6(2,0) free ? 8(2,2) free ?
  base station      decided, answered and logged in 0.011 ms
```

- One block per report, written by base_station.c as it happens
- "free ?" = no recent data on that node, so it is not known to be full
- Every run's full log is in logs/ in the submission zip

A8: Table 1, Tests and CAAS (Q&A)

logs/sample/node_004.log: spec Table 1, 10 s per cycle

Year	Month	Day	Hour	Minute	Second	Availability
2026	10	16	23	25	42	0
2026	10	16	23	25	52	0
2026	10	16	23	26	02	1
2026	10	16	23	26	12	2
2026	10	16	23	26	22	2
2026	10	16	23	26	32	2
2026	10	16	23	26	42	0
2026	10	16	23	26	52	0

- All 9 version pairs, 1x1, 1x8, 8x1, saturated, idle: clean exit
- ASan + UBSan: 0 errors
- Leaks: only inside MPI_Init_thread

- CAAS: caas/wsn_demo.job and caas/wsn_sweep.job, 2 nodes, --map-by node
- CAAS status: [update after the job has run, with the job id]
- Run it: make, then mpirun --oversubscribe -np 10 ./wsn 3 3
- Test evidence in tests/test_evidence.txt